DH2642 and ID2216 Intended Learning Outcomes: Having passed the course, the student will be able to

- choose appropriate technical platforms or JavaScript frameworks to create **highly-useable *data-persistent*** interactive web (DH2642) or native (ID2216) applications or native applications
- program interactive web applications according to **Model-View-Controller or related architectures**
- program systems that read data from, and send data to, **web APIs** with good **user experience**
- assess and **improve the usability** of existing interactive web applications
- **cooperate** with others to implement interactive web applications.

Criteria below are for the **project grade**. Individual bonus applies on top of that. **Late projects can reach max grade C**
To get to grade X, then minimum X is required on **all** criteria (or max if max<X for that criterion). **Aplus** on a criterion can compensate for small shortcomings in some other criterion. **UX Prize** gives extra bonus for well-designed and implemented user experience.

	Fx	E	D	C	B	A
Architecture/code **Separation of concerns:** 1. Persistence of application state 2. Application state (POJO model, Redux, Vuex, RxJS, Recoil, Mobx, Zustand…) 3. Presenter 4. View Navigation concern Keywords: Model-View-Presenter Model-view-controller **User-visible** third-party components (router, promise resolver/suspense do **not** count)	**Bugs** that prevent application usage Application state spread out in Views All 4 concerns mixed (e.g. View native events save to persistence…) One single view of application state UI components access data via **global** scope/imports rather than props	Views with multiple responsibilities/roles (e.g. mix Sidebar and Summary in one component) Multiple Views in a module (file) 3 **of** 4 concerns mixed (e.g. top-level state) No Application state (e.g. Presenters connected directly to Persistence)	Views with multiple responsibilities/roles One module per View. View-Presenter mix incidentally Application State mixed with Persistence but separated from Presenters and Views	Application state separated from all other concerns Views with separate and clear roles One module per View.	None of the four main concerns mixed (Persistence, Application state , Presenter/Controller, View) Views with separate and clear roles. One module per View. At least one **user-visible component written by others**	None of the four main concerns mixed (Persistence, Application state , Presenter/Controller, View) Views with separate and clear roles. One module per View. At least one **user-visible component written by others** **State manager** (Redux, Recoil, Pinia, Zustand, Mobx **with actions**). All application state side effects (e.g. persistence) use the state manager **A plus:** Framework independent Redux mappings (connect) instead of custom hooks

	Fx	E	D	C	B	A
Usability/User experience/ improve usability Keywords: Target group Use case Beyond state of art Effectiveness Efficiency Satisfaction Discoverable/explorable Feedback to action Visibility of status User in control User experience Prototyping Formative evaluation Expert evaluation	No feedback on user actions Poor visibility of system status No user consultation No improvement based on usability feedback	Effective application (a task can be accomplished) but the target group is not clear Inefficient user task accomplishment Little feedback on user actions Low visibility of system status No user consultation	Somewhat clear target group, unclear benefits for the user Moderately efficient task accomplishment User feels somewhat in control. Satisfactory feedback on user actions Satisfactory visibility of system status Little user consultation at prototyping or formative evaluation stage Some improvement based on usability feedback from users in the target group	Clear target group, clear benefits for the user Efficient task accomplishment Good feedback on user actions Good visibility of system status User feels in control. Moderate user satisfaction (pleasure of using the system) Documented user consultation, one 30 min session at prototyping **or** evaluation stage, documented improvements from user feedback	Clear target group, clear benefits for the user. Application functionality is easy to discover through exploration Efficient task accomplishment Very good feedback on user actions Very good visibility of system status User feels in control. Good user satisfaction (pleasure of using the system) Documented user consultation (30 min prototyping and 30 min formative evaluation, **or** 2x30min formative evaluation), with documented improvements from user feedback	Clear target group, clear benefits for the user. Application functionality is easy to discover through exploration Efficient task accomplishment. Very good feedback on user actions Very good visibility of system status User feels in control. Documented user consultation, at least 30 min at prototyping stage and 30 min at formative evaluation stage, with **creative** improvements from user feedback Very good user satisfaction **A plus, UX prize:** originality (app idea **beyond state of art**). **UX prize:** Very good user satisfaction **Responsive**/mobile-first design if it makes sense for the use case

	Fx	E	D	C	B	A
Web APIs and Persistence Keywords: Single vs multiple data source App state persistence (e.g. Firebase) User authentication (e.g. Firebase)	Not using any remote data Application state not persisted at the server Users can see each other's persisted data while (most cases) the application should keep it separate	Remote data is used from a single source, to make collections **and** summaries (**dinner planner in disguise**) Some system status shown when waiting for API requests Well separated persisted data, per **authenticated** user	Remote data is used from a single source but no dinner model in disguise! Create added value to users based on the data provided by the API Clear system status shown when waiting for API requests Well separated persisted data, per **authenticated** user	Remote data is used from a single source, but no dinner planner in disguise! Create added value to users based on the data provided by the API Clear system status shown when waiting for API requests and the user can choose to perform other actions while waiting for the response to arrive Well separated persisted data, per **authenticated** user Persisted data increases usability (e.g. efficiency user doesn't need to enter data again, users sees comparisons with other users' averages etc) **A plus:** live update of persistence. A user running multiple copies of the app sees the same data **A plus:** mashing-up data from several external sources/ APIs **A plus**: collaborative features on shared application state. User actions complement/help each other and user action conflicts are managed		
Group cooperation Keywords: Balance of work Role separation Individual self-reflection	Free riding, one or more persons do all work and one does not do anything or very little (**contact project coach** ASAP)	Good balance of work in the group Some role separation in the group (per component or per concern: views, interaction, model…) Work amount and roles documented through individual self-reflections	Good balance of work in the group Role separation in the group (per component or per concern: views, interaction, model/application state…) Work amount and roles well documented in self-reflections			